A Project Report

on

FLEET VEHICLE MANAGEMENT SYSTEM

JEEVITHASRI R **(927623BEC091)**

ABSTRACT

Fleet Vehicle Management System is a comprehensive software-based solution designed to monitor, manage, and optimize the operations of a fleet of vehicles used by organizations for transportation and logistics. The system integrates GPS tracking, real-time vehicle diagnostics, route optimization, fuel usage monitoring, driver behavior analysis, and maintenance scheduling. By leveraging cloud computing and Internet of Things (IoT) technologies, the system ensures effective fleet coordination, enhances operational efficiency, reduces fuel costs, and improves safety. Additionally, it provides a centralized platform for data collection and analytics, enabling fleet managers to make informed decisions and streamline day-to-day operations. This system is particularly beneficial for businesses involved in delivery services, logistics, public transportation, and field service operations

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1 OBJECTIVE

With the increasing demand for efficient transportation and logistics services, managing a large number of vehicles manually has become a challenging task for businesses. A Fleet Vehicle Management System provides an automated and intelligent approach to monitor, control, and optimize fleet operations. It enables real-time tracking of vehicles, analysis of driver behaviour, route planning, and timely maintenance scheduling, all from a centralized dashboard. By leveraging technologies like GPS, IoT, and cloud computing, this system enhances overall efficiency, reduces operational costs, and improves vehicle safety. The growing need for smarter logistics solutions makes such a system essential for companies involved in transportation, delivery services, and field operations.

1.2 OVERVIEW

The Fleet Vehicle Management System is a centralized platform designed to oversee and streamline all aspects of fleet operations. It incorporates features like real-time vehicle tracking, driver performance monitoring, route optimization, fuel usage analysis, and maintenance alerts. The system collects data from GPS and onboard sensors, processes it using cloud-based services, and presents it to fleet managers through an intuitive interface. This helps reduce fuel consumption, improve safety, ensure timely deliveries, and lower operational costs. The system is scalable and can be adapted to fleets of various sizes, making it suitable for logistics, transportation, and service-based industries.

1.3 DATABASE MANAGEMENT SYSTEMS' CONCEPTS

1.3.1 ENTITY – RELATIONSHIP (ER) MODEL

The **ER model**, which visually represents entities (e.g., Vehicle, Driver, Trip) and their relationships. This model helps in the logical design of the database before implementation. Entities are represented as rectangles, attributes as ovals, and relationships as diamonds or labeled connectors.

1.3.2 RELATIONAL MODEL

The relational model organizes data into tables (relations). Each table has rows (tuples) and columns (attributes). Relationships between tables are established using primary and foreign keys. For example:

- vehicle_id in Vehicle is a primary key.
- vehicle_id in Trip is a foreign key referencing Vehicle.

1.3.3 NORMALIZATION

Normalization is a process to organize data to reduce redundancy and improve data integrity. Our project uses:

- 1NF: No repeating groups; atomic values.
- 2NF: No partial dependency on a composite primary key.
- 3NF: No transitive dependency between non-key attributes. Example: Driver and License are split into separate tables to avoid data duplication.

1.3.4 PRIMARY KEY AND FOREIGN KEY

- Primary Key: Uniquely identifies each record in a table (e.g., trip_id, driver_id).
- Foreign Key: Establishes a link between two tables (e.g., driver_id in Trip references Driver).

1.3.5 INDEXING

Indexes are used to speed up search queries. Commonly indexed fields include:

- driver_id
- vehicle_id
- trip_id

They improve the performance of retrieval operations especially in large datasets.

1.3.6 SQL(STRUCTURED QUERY LANGUAGE)

SQL is used to interact with the database:

- SELECT to retrieve data
- INSERT to add new records
- UPDATE to modify data
- DELETE to remove data

For example: SELECT * FROM Trip WHERE driver_id = 101;

1.3.7 TRANSACTIONS AND ACID PROPERTIES

A transaction is a set of operations performed as a single unit. It must follow ACID:

- Atomicity: All or none of the operations occur.
- Consistency: Database remains in a valid state.
- Isolation: Transactions don't interfere.
- Durability: Once committed, changes are permanent. Example: Adding a new trip and updating vehicle status should happen together.

1.3.8 DATA INTEGRITY

Ensures accuracy and consistency of data over its lifecycle. Enforced through:

- Constraints (e.g., NOT NULL, UNIQUE)
- Validation rules

1.3.9 SECURITY AND ACCESS CONTROL

Access control ensures only authorized users can access or modify data. In the fleet system:

- Admins have full access.
- Mechanics may access only maintenance data.

This is enforced using user roles and permissions.

1.3. BACKUP AND RECOVERY

To prevent data loss from failures, periodic backups are essential. Recovery procedures help to restore the system using backup files to maintain business continuity.

CHAPTER 2
PROJECT METHODOLOGY

2.1 PROPOSED WORK

The proposed Fleet Vehicle Management System is designed to automate the monitoring and management of vehicles, drivers, trips, licenses, maintenance, fuel logs, and associated personnel. The methodology begins with identifying the core requirements and designing an Entity-Relationship (ER) diagram that captures the relationships among entities like Vehicle, Driver, Trip, License, Mechanic, and Maintenance. The ER model is converted into a normalized relational schema to reduce redundancy and ensure data integrity using primary and foreign keys. A relational database management system (such as MySQL or PostgreSQL) is used to implement the backend, where CRUD operations are applied to manage records efficiently. Proper constraints and validation checks are introduced to maintain accuracy, such as license expiry dates, vehicle availability, and maintenance schedules.

Once the database is developed, all modules are integrated so that data flows consistently across the system like updating driver and vehicle status during trip assignment. SQL queries are used to generate useful reports on trip history, fuel usage, and maintenance costs. The system is thoroughly tested using unit and integration testing to ensure each module functions as expected. After testing, the system is deployed in a test or real-time environment, where its performance and usability are evaluated. The end goal is to build a robust, real-time system that minimizes manual effort, reduces errors, and helps fleet managers make informed operational decisions.

2.2 ARCHITECTURE

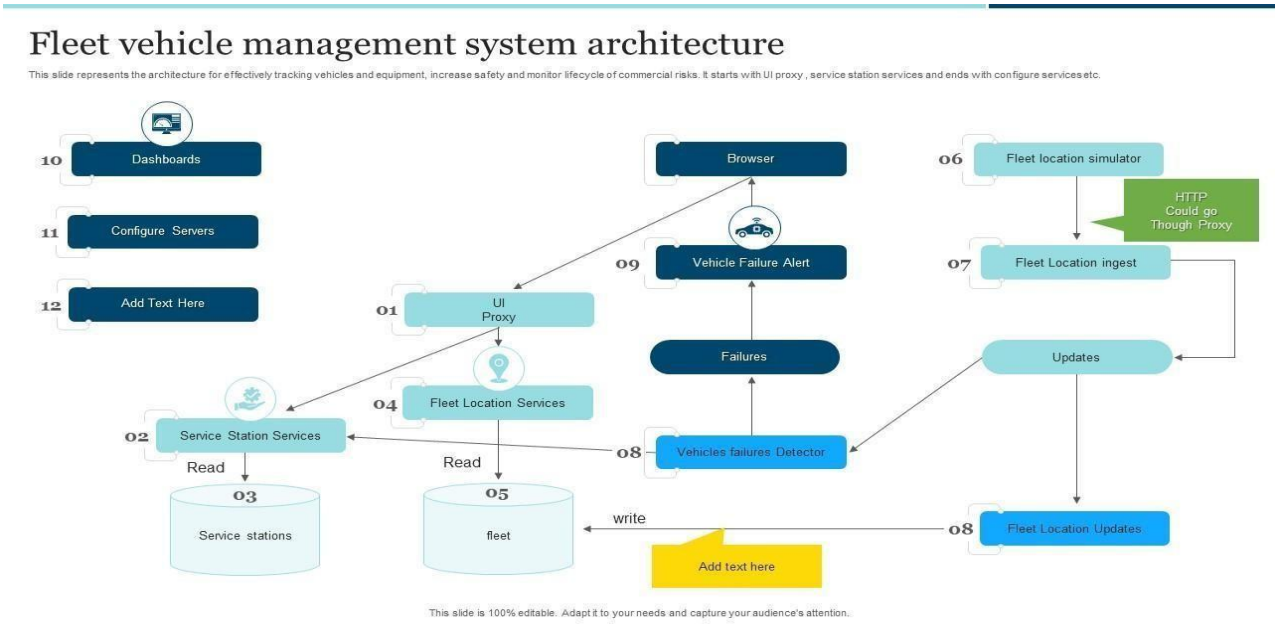

Fig 2.2.1

- The system begins with a UI Proxy that connects user interfaces to backend services.
- Fleet Location Services and Service Station Services fetch real-time data from fleet and service station databases.
- A Fleet Location Simulator generates vehicle movement data, which is processed by the Location Ingest module.
- This data is stored, updated, and displayed via dashboards and configuration interfaces.

  Alert system.
- All components work together to enable effective tracking, maintenance, and operational control of fleet vehicles.

2.3 E-R DIAGRAM

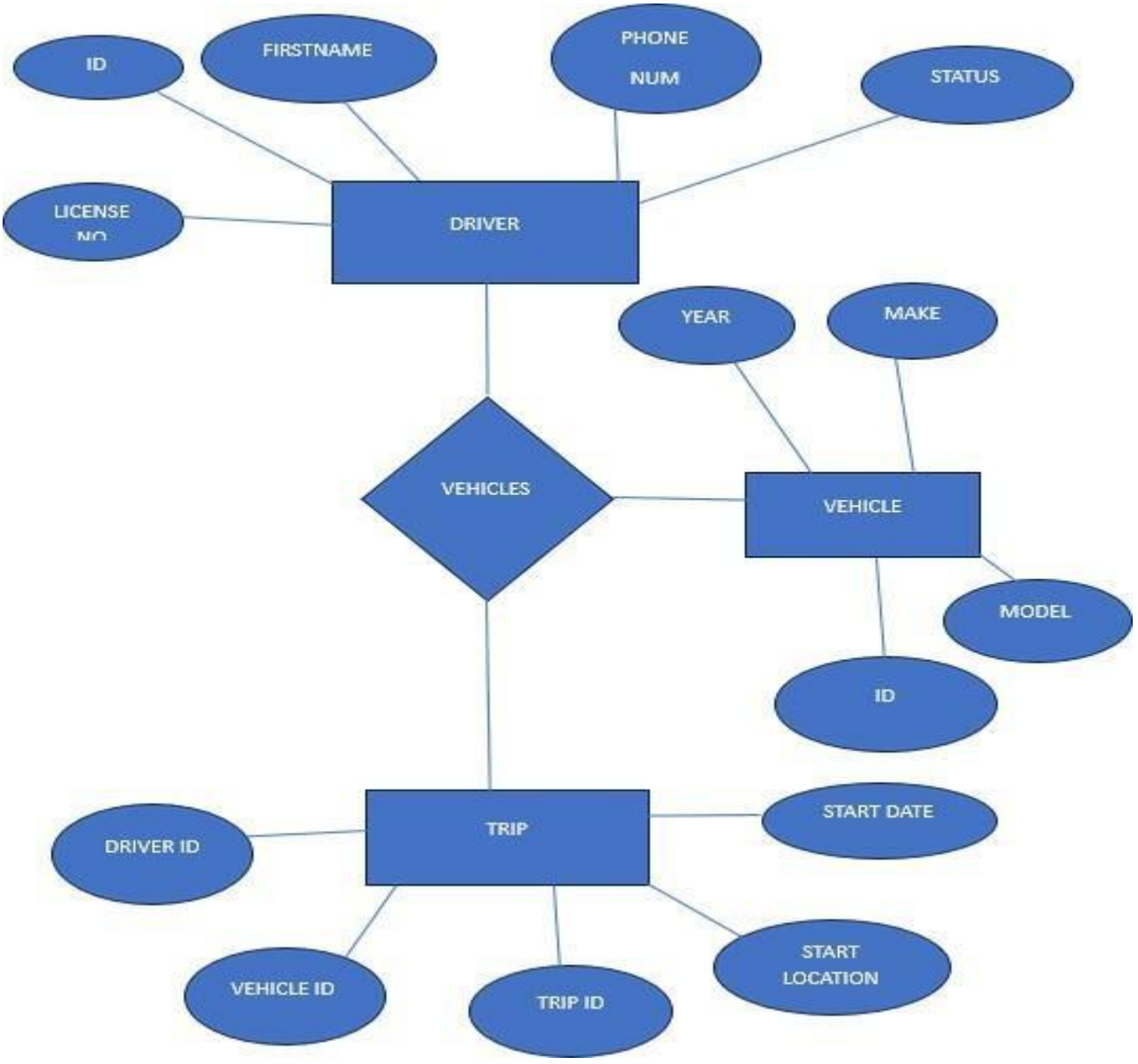

Fig 2.3.1`

Entities and Their Attributes:

1. Driver
    1. Attributes: ID, Firstname, Phone Num, Status, License No
    2. Represents the driver who operates the vehicle.
2. Vehicle
    1. Attributes: ID, Make, Model, Year
    2. Represents a vehicle in the fleet with its basic details.
3. Trip
    1. Attributes: Trip ID, Driver ID, Vehicle ID, Start Date, Start Location
    2. Represents a journey or task assigned to a driver and vehicle.

Relationships:

- VEHICLES (between DRIVER and VEHICLE)
    1. Indicates that a driver is assigned to one or more vehicles.
    2. Many-to-Many relationship is implied here (a driver can drive multiple vehicles and a vehicle can have multiple drivers over time).
- TRIP
    1. Connects a Driver and a Vehicle through the Driver ID and Vehicle ID.
    2. Helps track which driver took which vehicle, when, and from where.

CHAPTER 3
MODULE DESCRIPTION

3.1 User Interface Module

- A welcome page to collect customer details like name and contact number.
- A form to input trip-related data such as vehicle name, driver name, trip date, pick-up point, and destination.
- Clear, user-friendly navigation and form validation using JavaScript.
- Built using HTML and CSS to ensure responsiveness and accessibility.

3.2 Vehicle and Driver Management Module

This module manages the details of available vehicles and drivers. It includes:

- Storage and retrieval of vehicle names and associated properties.
- Driver details like name, availability, and assignment tracking.
- Helps in assigning appropriate drivers to vehicles based on trip needs.
- Ensures updated records for better trip scheduling.

3.2.1 Trip Scheduling Module

This module allows users to schedule a trip with specific inputs:

- Accepts trip date, pick-up point, and destination.
- Validates entries to avoid conflicting or invalid trip data.
- Connects driver and vehicle data to each trip.
- Can later be enhanced to detect overlaps or route optimization.

3.3 Database Management Module

This module focuses on integrating with a backend database:

- Stores all customer, vehicle, driver, and trip data securely.
- Performs CRUD (Create, Read, Update, Delete) operations.
- Ensures consistency and data integrity across modules.
- Allows easy data retrieval for admin or reporting purposes.

3.4 Report & Confirmation Module

This module generates confirmation and reports of trips:

- Shows a summary or confirmation after a trip is submitted.
- Can be extended to email or SMS confirmations.
- Generates logs of trips completed and pending.
- Useful for administrative review and analytics in the future.

CHAPTER 4

RESULTS AND DISCUSSION

WELCOME PAGE :

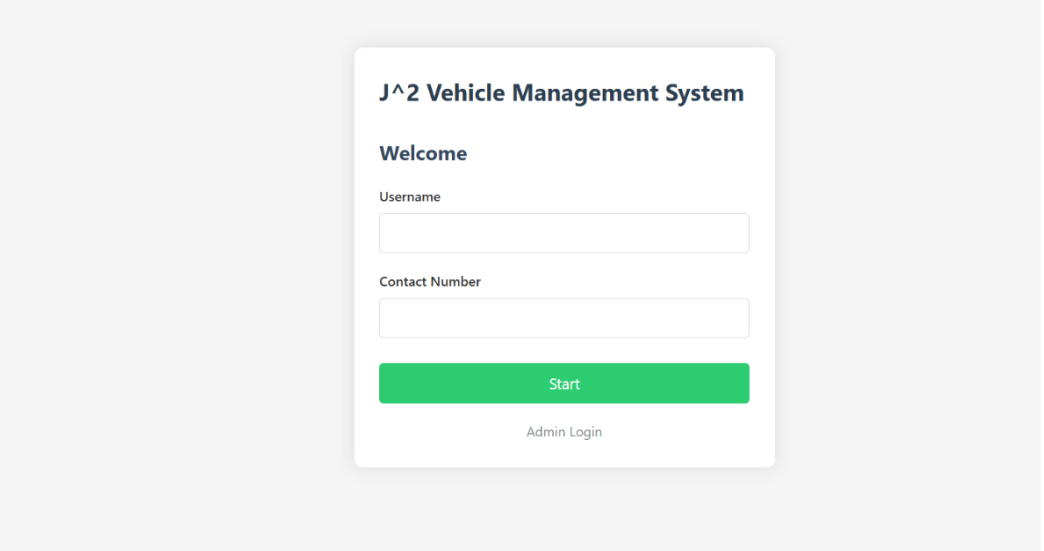

Fig 4.1

The Welcome Page of the Fleet Vehicle Management System acts as the entry point for users interacting with the platform. It prominently features the company name "J²" at the top, giving a branded feel to the interface.

Below the title, users are prompted to enter:

- Customer Name – to personalize the experience and track user interactions.
- Contact Number – essential for follow-up and communication.

A “Proceed” button is provided to take the user to the next section of the system, where trip details can be entered. The page uses a clean, centred layout with friendly fonts and soft background colours to create a welcoming and professional look.

TRIP DETAIL FORM :

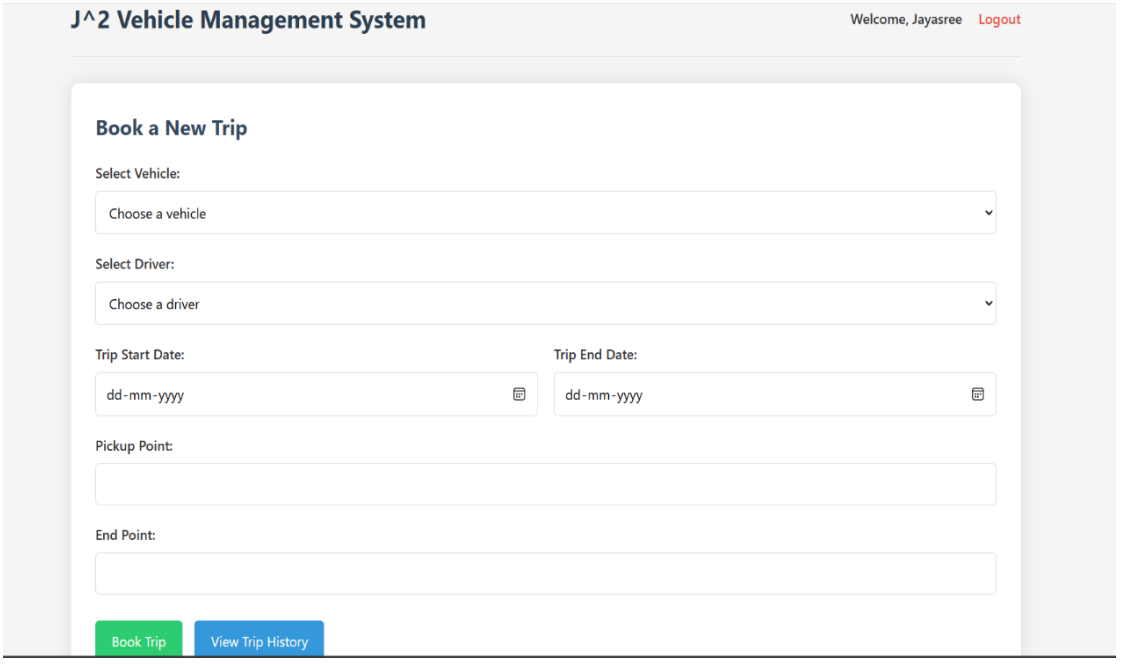

Fig 4.2

- A Trip detail form that allows entry of
  - Vehicle name
  - Driver name (from a selectable list)
  - Trip date
  - Pickup point
  - Destination

The design uses intuitive input fields and organized labels to ensure smooth interaction. Once the form is submitted, data is sent to the backend for processing and storage. The interface is responsive and styled with modern UI elements for a better user experience.

TRIP HISTORY :

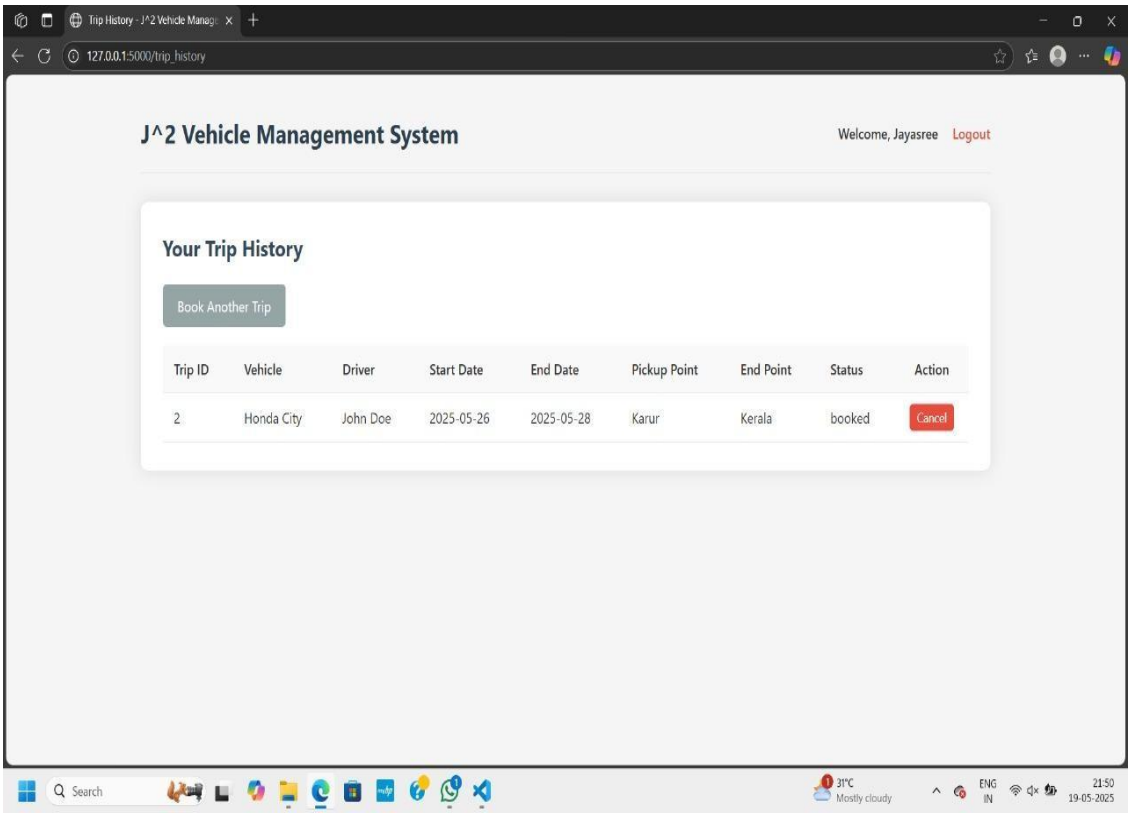

Fig 4.3

The image shows the **Trip History page** of the **J^2 Vehicle Management System**, where users can view details of their booked trips. At the top, it greets the logged-in user (e.g., "Welcome, Jayasree") and provides a logout option for secure exit. The main section displays the heading "Your Trip History" and includes a button to book a new trip. Below this, a table lists trip details such as Trip ID, Vehicle, Driver, Start and End Dates, Pickup Point, End Point, and Status. It also provides an action button to cancel a trip if needed. This page helps users manage and track their vehicle bookings effectively.

CHAPTER 5
CONCLUSION

The Fleet Vehicle Management System provides an efficient, centralized solution for monitoring, maintaining, and managing fleet operations. By integrating vehicle tracking, maintenance scheduling, fuel management, and driver monitoring, the system enhances operational efficiency, reduces costs, and improves safety and compliance. Automation of routine tasks and real-time data access empower fleet managers to make informed decisions quickly. Overall, the implementation of this system contributes significantly to optimizing resource utilization, minimizing downtime, and ensuring a more reliable and productive fleet operation.

REFERENCES

[1] Patel, R., & Mehta, S. (2020). A Smart Fleet Management System Using GPS and GSM. International Journal of Scientific Research in Engineering and Management (IJSREM), 4(8), 23-29.
– Discusses the integration of GPS and GSM for real-time vehicle tracking and alerts.

[2] Kumar, V., & Singh, A. (2021). Design and Implementation of Vehicle Tracking System Using IoT. Journal of Emerging Technologies and Innovative Research (JETIR), 8(3), 66-72.
– Explores IoT-based systems for vehicle management and trip monitoring.

[3] Bharathi, V., & Rajkumar, S. (2019). Fleet Management Using Database Systems. International Journal of Computer Applications, 178(5),12-18.
– Focuses on DBMS-based solutions for storing and querying trip, driver, and vehicle data.

[4] Zhang, Y., & Tan, W. (2018). Optimizing Fleet Routing and Scheduling Using Software Systems. Transportation Research Procedia, 32,142–149.
– Analyse the impact of routing algorithms on fleet efficiency.

[5] Mohan, K., & Saran, R. (2022). Web-Based Fleet Management Application Using MERN Stack. International Journal of Advanced Research in Computer Science, 13(2), 90-97.
– Discusses a full-stack web app (MongoDB, Express.js, React, Node.js) for managing vehicle and trip information

APPENDIX

(Coding)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Admin - J^2 Vehicle Management System</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/admin.css') }}">
</head>
<body>
    <div class="container">
        <header>
            <h1>J^2 Vehicle Management System - Admin</h1>
            {% if dashboard %}
            <div class="admin-info">
                <a href="{{ url_for('admin_logout') }}" class="logout-btn">Logout</a>
            </div>
            {% endif %}
        </header>

        <div class="content">
            {% if not dashboard %}
            <div class="admin-login-container">
                <h2>Admin Login</h2>
                <form action="{{ url_for('admin_login') }}" method="post">
                    <div class="form-group">
                        <label for="username">Username</label>
                        <input type="text" id="username" name="username" required>
                    </div>
                    <div class="form-group">
                        <label for="password">Password</label>
                        <input type="password" id="password" name="password" required>
                    </div>
                    <button type="submit" class="btn login-btn">Login</button>
                </form>
                <div class="back-link">
                    <a href="{{ url_for('index') }}">Back to User Login</a>
                </div>
            </div>
            {% else %}
            <div class="admin-dashboard">
                <div class="admin-tabs">
                    <ul class="tab-links">
```

```
                <li><a href="#bookings" class="tab-link active" onclick="openTab(event,
'bookings')">Bookings</a></li>
                <li><a href="#vehicles" class="tab-link" onclick="openTab(event,
'vehicles')">Vehicles</a></li>
                <li><a href="#drivers" class="tab-link" onclick="openTab(event,
'drivers')">Drivers</a></li>
            </ul>
        </div>

        <div id="bookings" class="tab-content active">
            <h2>All Bookings</h2>
            {% if trips %}
            <div class="trips-table">
                <table>
                    <thead>
                        <tr>
                            <th>Trip ID</th>
                            <th>User</th>
                            <th>Contact</th>
                            <th>Vehicle</th>
                            <th>Driver</th>
                            <th>Start Date</th>
                            <th>End Date</th>
                            <th>Pickup Point</th>
                            <th>End Point</th>
                            <th>Status</th>
                            <th>Action</th>
                        </tr>
                    </thead>
                    <tbody>
                        {% for trip in trips %}
                        <tr>
                            <td>{{ trip.id }}</td>
                            <td>{{ trip.username }}</td>
                            <td>{{ trip.contact }}</td>
                            <td>{{ trip.vehicle_name }}</td>
                            <td>{{ trip.driver_name }}</td>
                            <td>{{ trip.start_date }}</td>
                            <td>{{ trip.end_date }}</td>
                            <td>{{ trip.pickup_point }}</td>
                            <td>{{ trip.end_point }}</td>
                            <td>{{ trip.status }}</td>
                            <td>
                                {% if trip.status != 'cancelled' %}
                                <form action="{{ url_for('admin_cancel_trip', trip_id=trip.id) }}"
method="post" class="inline-form">
```

```
                                <button type="submit" class="btn delete-btn" onclick="return
confirm('Are you sure you want to cancel this trip?')">Cancel</button>
                            </form>
                                {% endif %}
                            </td>
                        </tr>
                        {% endfor %}
                    </tbody>
                </table>
            </div>
            {% else %}
            <div class="no-trips">
                <p>No bookings found in the system.</p>
            </div>
            {% endif %}
        </div>

        <div id="vehicles" class="tab-content">
            <div class="add-section">
                <h2>Add New Vehicle</h2>
                <form action="{{ url_for('add_vehicle') }}" method="post">
                    <div class="form-row">
                        <div class="form-group">
                            <label for="vehicle-name">Vehicle Name</label>
                            <input type="text" id="vehicle-name" name="name" required>
                        </div>
                        <div class="form-group">
                            <label for="vehicle-model">Model</label>
                            <input type="text" id="vehicle-model" name="model" required>
                        </div>
                        <div class="form-group">
                            <label for="vehicle-capacity">Capacity</label>
                            <input type="number" id="vehicle-capacity" name="capacity" min="1"
required>
                        </div>
                        <div class="form-group">
                            <button type="submit" class="btn add-btn">Add Vehicle</button>
                        </div>
                    </div>
                </form>
            </div>

            <h2>Vehicle List</h2>
            {% if vehicles %}
            <div class="vehicles-table">
                <table>
                    <thead>
                        <tr>
```

```
                        <th>ID</th>
                        <th>Name</th>
                        <th>Model</th>
                        <th>Capacity</th>
                        <th>Action</th>
                    </tr>
                </thead>
                <tbody>
                    {% for vehicle in vehicles %}
                    <tr>
                        <td>{{ vehicle.id }}</td>
                        <td>{{ vehicle.name }}</td>
                        <td>{{ vehicle.model }}</td>
                        <td>{{ vehicle.capacity }}</td>
                        <td>
                            <form action="{{ url_for('delete_vehicle', vehicle_id=vehicle.id) }}"
method="post" class="inline-form">
                                <button type="submit" class="btn delete-btn" onclick="return
confirm('Are you sure you want to delete this vehicle?')">Delete</button>
                            </form>
                        </td>
                    </tr>
                    {% endfor %}
                </tbody>
            </table>
        </div>
        {% else %}
        <div class="no-vehicles">
            <p>No vehicles found in the system.</p>
        </div>
        {% endif %}
    </div>

    <div id="drivers" class="tab-content">
        <div class="add-section">
            <h2>Add New Driver</h2>
            <form action="{{ url_for('add_driver') }}" method="post">
                <div class="form-row">
                    <div class="form-group">
                        <label for="driver-name">Driver Name</label>
                        <input type="text" id="driver-name" name="name" required>
                    </div>
                    <div class="form-group">
                        <label for="driver-experience">Experience (Years)</label>
                        <input type="number" id="driver-experience" name="experience" min="0"
required>
```

```
                    </div>
                    <div class="form-group">
                        <button type="submit" class="btn add-btn">Add Driver</button>
                    </div>
                </div>
            </form>
        </div>

        <h2>Driver List</h2>
        {% if drivers %}
        <div class="drivers-table">
            <table>
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Experience (Years)</th>
                        <th>Action</th>
                    </tr>
                </thead>
                <tbody>
                    {% for driver in drivers %}
                    <tr>
                        <td>{{ driver.id }}</td>
                        <td>{{ driver.name }}</td>
                        <td>{{ driver.experience }}</td>
                        <td>
                            <form action="{{ url_for('delete_driver', driver_id=driver.id) }}"
method="post" class="inline-form">
                                <button type="submit" class="btn delete-btn" onclick="return
confirm('Are you sure you want to delete this driver?')">Delete</button>
                            </form>
                        </td>
                    </tr>
                    {% endfor %}
                </tbody>
            </table>
        </div>
        {% else %}
        <div class="no-drivers">
            <p>No drivers found in the system.</p>
        </div>
        {% endif %}
    </div>
</div>
{% endif %}
</div>
</div>
```

```
    <script src="{{ url_for('static', filename='js/admin.js') }}"></script>
</body>
</html>
```